

Week 2 Lab

Flowchart Practice: Decisions + Loops, Together

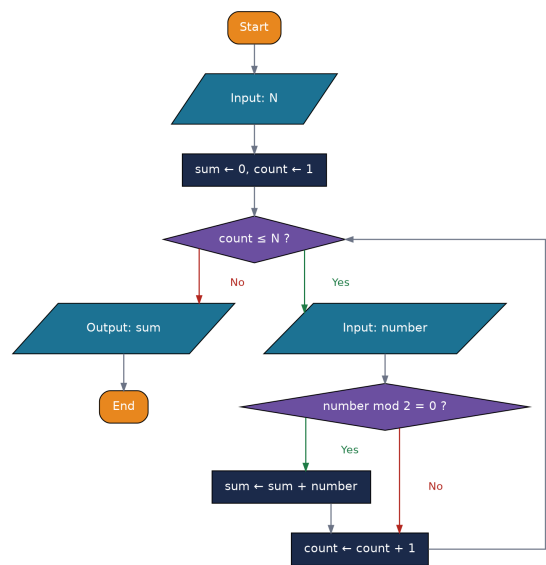
DELIVERED BY:	Teaching Assistants
FOLLOWS:	Lecture — Decisions + Loops, Together
FORMAT:	Pen and paper, or a diagramming tool — no Java code in this lab

GROUND RULES

- Same four symbols as last week: Terminal (Start/End), Process, Decision, Input/Output.
- Questions get progressively harder: conditional accumulation → running best → nested loops → validation → search → a capstone that combines everything.
- Every Decision diamond needs exactly two labeled exits: Yes and No.
- Each question's flowchart, pseudocode, and trace table are all on one page, right after the question. Cover them while solving.

Q1. Input N numbers; sum only the even ones.

FLOWCHART



PSEUDOCODE

```
1. input N
2. sum ← 0, count ← 1
3. while count ≤ N do
4.   input number
5.   if number mod 2 = 0 then
6.     sum ← sum + number
7.   end if
8.   count ← count + 1
9. end while
10. output sum
```

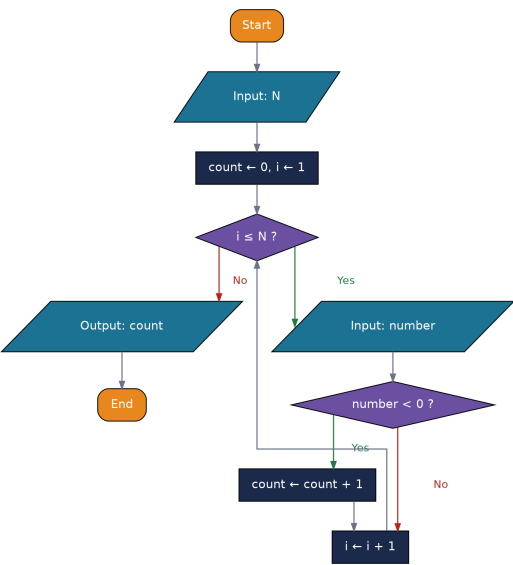
TRACE TABLE

Sample run with N = 5, inputs: 3, 8, 5, 12, 7.

count	number	%2=0?	sum
1	3	No	0
2	8	Yes	0+8=8
3	5	No	8
4	12	Yes	8+12=20
5	7	No	20
6	—	exit	output 20

Q2. Input N numbers; count how many are negative.

FLOWCHART



PSEUDOCODE

```
1. input N
2. count ← 0, i ← 1
3. while i ≤ N do
4.   input number
5.   if number < 0 then
6.     count ← count + 1
7.   end if
8.   i ← i + 1
9. end while
10. output count
```

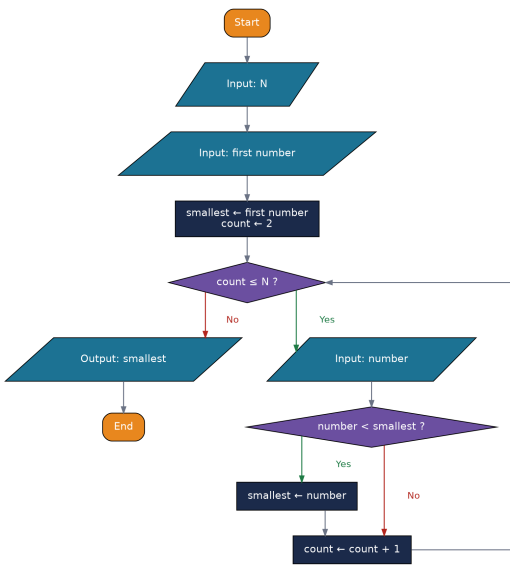
TRACE TABLE

Sample run with N = 5, inputs: -3, 4, -7, 2, -1.

i	number	< 0 ?	count
1	-3	Yes	1
2	4	No	1
3	-7	Yes	2
4	2	No	2
5	-1	Yes	3
6	—	exit	output 3

Q3. Input N numbers; find the smallest one.

FLOWCHART



PSEUDOCODE

```
1. input N
2. input first number
3. smallest ← first number, count ← 2
4. while count ≤ N do
5.   input number
6.   if number < smallest then
7.     smallest ← number
8.   end if
9.   count ← count + 1
10. end while
11. output smallest
```

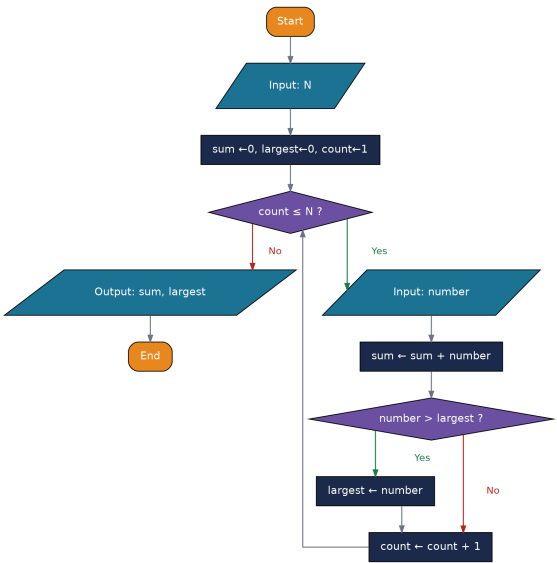
TRACE TABLE

Sample run with N = 5: first number is 8, then 3, 15, 1, 9.

count	number	<smallest?	smallest
—	8 (first)	—	8
2	3	Yes	3
3	15	No	3
4	1	Yes	1
5	9	No	1
6	—	exit	output 1

Q4. Input N numbers; output both their sum and the largest one, in a single pass.

FLOWCHART



PSEUDOCODE

```
1. input N
2. sum ← 0, largest ← 0, count ← 1
3. while count ≤ N do
4.   input number
5.   sum ← sum + number
6.   if number > largest then
7.     largest ← number
8.   end if
9.   count ← count + 1
10. end while
11. output sum, largest
```

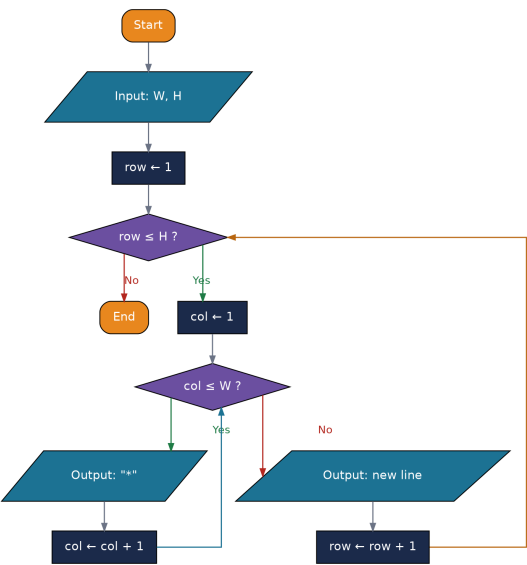
TRACE TABLE

Sample run with N = 4, inputs: 5, 9, 2, 7.

count	number	sum	>largest?	largest
1	5	5	Yes	5
2	9	14	Yes	9
3	2	16	No	9
4	7	23	No	9
5	—	—	exit	output 23, 9

Q5. Input W and H; print a rectangle of stars — H rows of W stars each.

FLOWCHART



PSEUDOCODE

```
1. input W, H
2. row ← 1
3. while row ≤ H do
4.   col ← 1
5.   while col ≤ W do
6.     output "*"
7.     col ← col + 1
8.   end while
9.   output new line
10.  row ← row + 1
11. end while
```

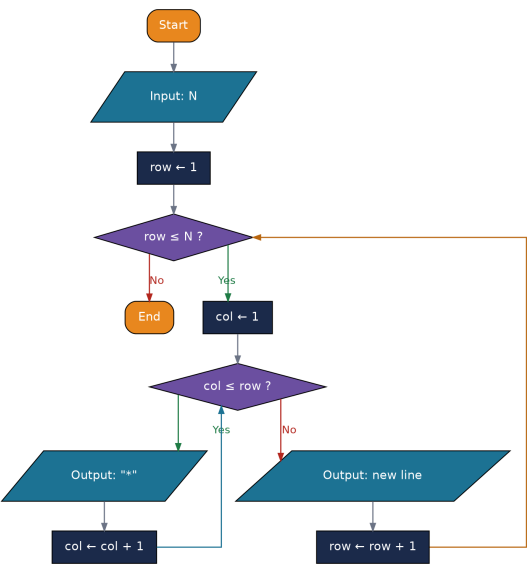
TRACE TABLE

Sample run with W = 4, H = 3 — every row prints the same fixed width.

row	stars printed	row's output
1	4	****
2	4	****
3	4	****
4	—	exit (row > H)

Q6. Input N; print a right triangle of stars — row i has exactly i stars.

FLOWCHART



PSEUDOCODE

```
1. input N
2. row ← 1
3. while row ≤ N do
4.   col ← 1
5.   while col ≤ row do
6.     output "*"
7.     col ← col + 1
8.   end while
9.   output new line
10.  row ← row + 1
11. end while
```

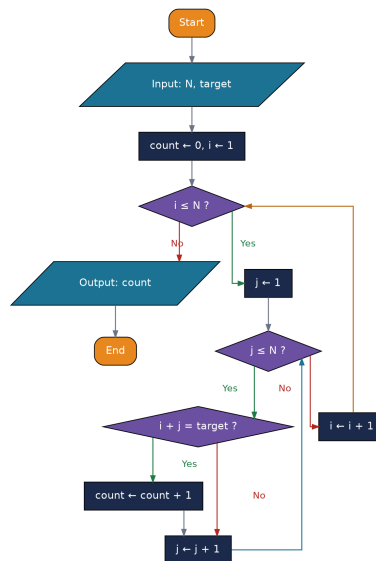
TRACE TABLE

Sample run with N = 4 — unlike Q5, each row's width now depends on row itself.

row	stars printed	row's output
1	1	*
2	2	**
3	3	***
4	4	****
5	—	exit (row > N)

Q7. Input N and a target; count how many pairs (i, j), each from 1 to N, add up to target.

FLOWCHART



PSEUDOCODE

```

1. input N, target
2. count ← 0, i ← 1
3. while i ≤ N do
4.   j ← 1
5.   while j ≤ N do
6.     if i + j = target then
7.       count ← count + 1
8.     end if
9.     j ← j + 1
10.  end while
11.  i ← i + 1
12. end while
13. output count
  
```

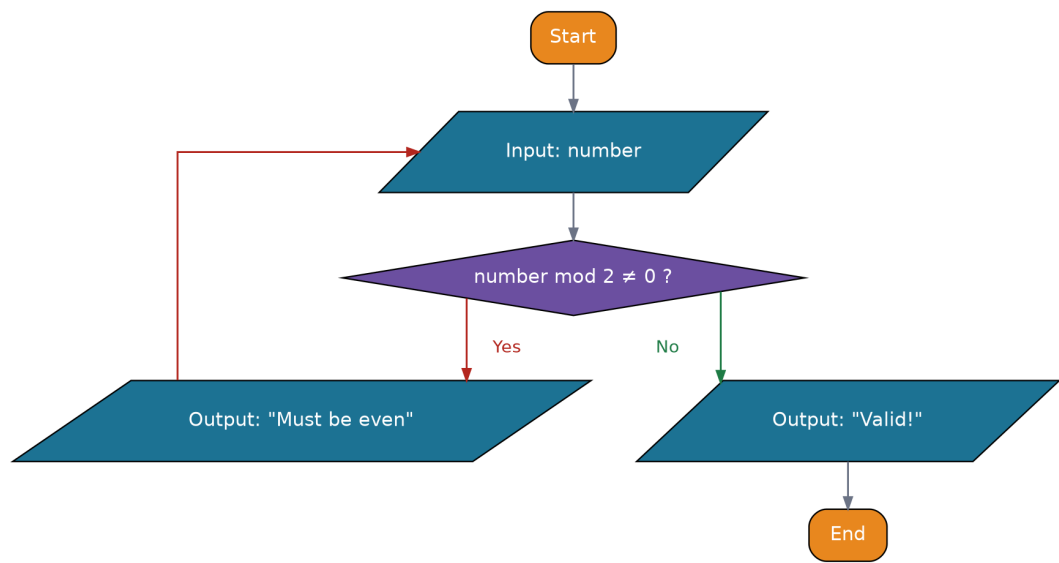
TRACE TABLE

Sample run with $N = 3$, $\text{target} = 4$.

i	j	i+j	=target?	count
1	1	2	No	0
1	2	3	No	0
1	3	4	Yes	1
2	1	3	No	1
2	2	4	Yes	2
2	3	5	No	2
3	1	4	Yes	3
3	2	5	No	3
3	3	6	No	3
exit	—	—	—	output 3

Q8. Keep asking for a number until an even one is entered.

FLOWCHART



PSEUDOCODE

```
1. input number
2. while number mod 2 ≠ 0 do
3.   output "Must be even"
4.   input number
5. end while
6. output "Valid!"
```

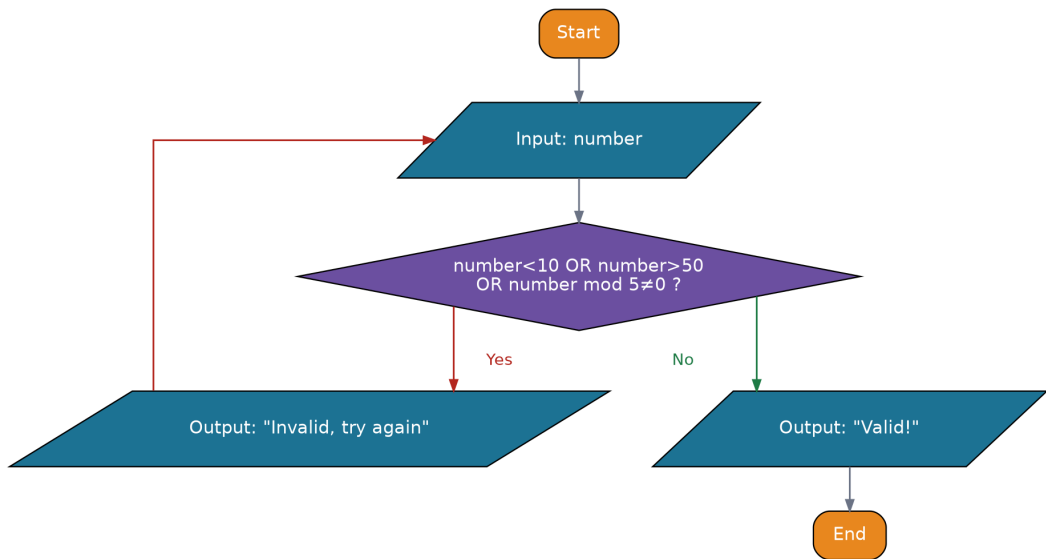
TRACE TABLE

Sample run with inputs 7, 10.

pass	number	mod 2 ≠ 0 ?	action
1	7	Yes	ask again
2	10	No	accepted!

Q9. Keep asking for a number until it is between 10 and 50 (inclusive) AND a multiple of 5.

FLOWCHART



PSEUDOCODE

```
1. input number
2. while number < 10 OR number > 50
3.   OR number mod 5 ≠ 0 do
4.   output "Invalid, try again"
5.   input number
6. end while
7. output "Valid!"
```

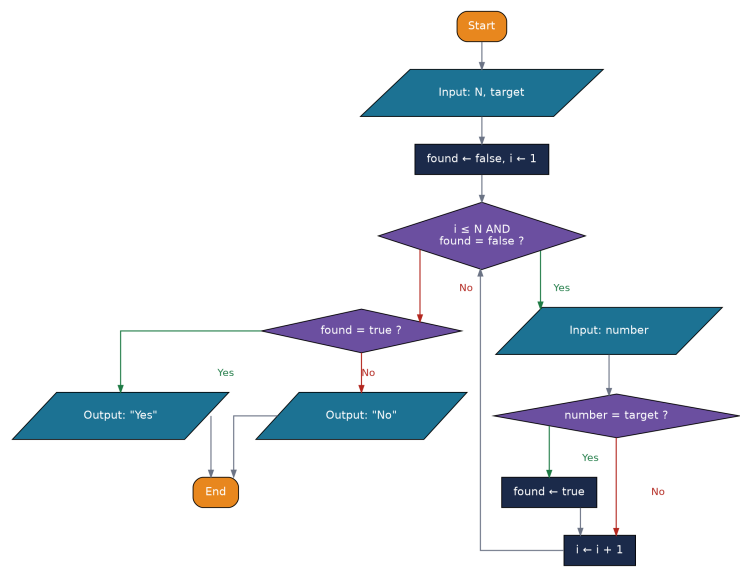
TRACE TABLE

Sample run with inputs 7, 63, 22, 25.

pass	number	invalid?	why
1	7	Yes	< 10
2	63	Yes	> 50
3	22	Yes	not ×5
4	25	No	accepted!

Q10. Input N numbers and a target; output whether the target appears among them (Yes/No).

FLOWCHART



PSEUDOCODE

```
1. input N, target
2. found ← false, i ← 1
3. while i ≤ N AND found = false do
4.   input number
5.   if number = target then
6.     found ← true
7.   end if
8.   i ← i + 1
9. end while
10. if found = true then output "Yes"
11. else output "No"
```

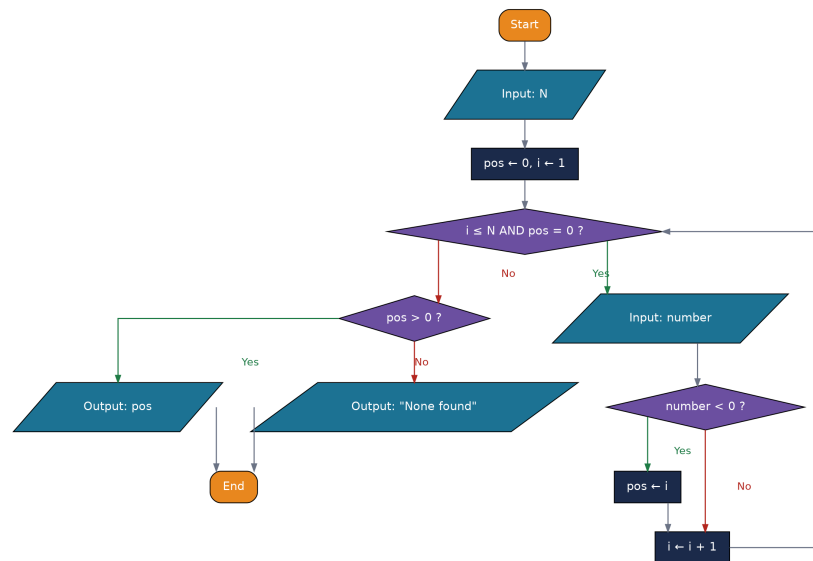
TRACE TABLE

Sample run with N = 5, target = 7, inputs: 3, 9, 2, 7, 5.

i	continue?	number	=target?	found
1	Yes	3	No	false
2	Yes	9	No	false
3	Yes	2	No	false
4	Yes	7	Yes	true
5	No (found)	—	—	exit – output "Yes"

Q11. Input N numbers; output the position of the first negative one (or “None found”).

FLOWCHART



PSEUDOCODE

```

1. input N
2. pos ← 0, i ← 1
3. while i ≤ N AND pos = 0 do
4.   input number
5.   if number < 0 then
6.     pos ← i
7.   end if
8.   i ← i + 1
9. end while
10. if pos > 0 then output pos
11. else output "None found"

```

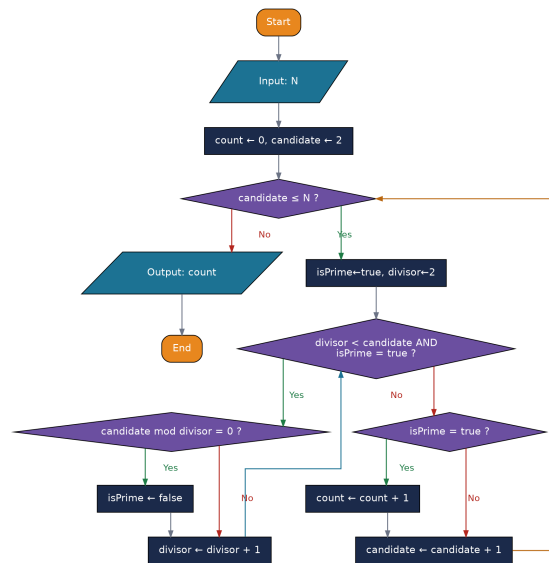
TRACE TABLE

Sample run with $N = 5$, inputs: 4, 7, -2, 9, -1.

i	continue?	number	< 0 ?	pos
1	Yes	4	No	0
2	Yes	7	No	0
3	Yes	-2	Yes	3
4	No (pos≠0)	—	—	exit — output 3

Q12. Input N; count how many whole numbers from 2 to N are prime (divisible by nothing except 1 and themselves).

FLOWCHART



PSEUDOCODE

```

1. input N
2. count ← 0, candidate ← 2
3. while candidate ≤ N do
4.   isPrime ← true, divisor ← 2
5.   while divisor < candidate AND isPrime = true do
6.     if candidate mod divisor = 0 then
7.       isPrime ← false
8.     end if
9.     divisor ← divisor + 1
10.  end while
11.  if isPrime = true then count ← count + 1
12.  candidate ← candidate + 1
13. end while
14. output count
  
```

Three patterns in one: a loop inside a loop, an early-stop flag (*isPrime*), and a running count.

TRACE TABLE

Sample run with $N = 7$ — the primes found are 2, 3, 5, 7.

candidate	isPrime?	count
2	true	1
3	true	2
4	false (2 divides it)	2
5	true	3
6	false (2 divides it)	3
7	true	4
exit (candidate=8)	—	output 4

WRAP-UP & INSTRUCTOR NOTES

Wrap-up check (5 min)

Before leaving, each student should be able to point to their Q6 or Q7 flowchart and answer out loud:

- “Which loop is the outer one, and which is the inner one?”
- “How many times does the innermost step run, in total?”

Instructor notes

- Do not let students write Java for these — the point is separating logic from syntax. Redirect anyone who starts typing code back to the diagram.
- Q3 and Q4 look similar — both track a running value — but Q4 is harder: it updates sum unconditionally while largest only updates sometimes, in the same pass.
- Q5, Q6, and Q7 build in difficulty on purpose: Q5 is a fixed-size rectangle (easiest), Q6 changes the inner bound to depend on the outer counter (row), and Q7 adds a decision inside the inner loop as well. Watch for students who forget that the inner loop's counter must reset every time the outer loop takes a step.
- Q10 and Q11 are both search, but Q10 only asks yes/no while Q11 asks where — returning a position needs one more piece of bookkeeping (pos) than a simple flag does.
- Q12 is a genuine capstone — nested loop, early-stop flag, and accumulation all at once. Most students will need real time on it; treat it as a stretch goal, not a baseline expectation.
- The diagrams, pseudocode, and traces in this packet show one correct shape, not the only one — a student's version with a different but logically equivalent order is still correct.
- Next lecture: translating these exact flowcharts into Java if/else and while/for syntax — keep today's diagrams, students will reuse them.